



Tecnológico de Monterrey

Campus Guadalajara

Reto Selección, configuración y entrenamiento del Modelo

Miguel Ángel Rivas Torres | A01644628

Alejandra Gonzalez Burnes | A01286272

Jose Yael Varela Garcia | A01645324

11 de 09 del 2026

1. Introducción

El reto consiste en predecir a qué ejercicio de rehabilitación (`movement_id`) corresponde una repetición registrada por los sensores, a partir de las features calculadas sobre esa señal. `movement_id` es una variable categórica con 15 valores posibles (ver sección 3), por lo tanto el problema es de clasificación multiclase, no de regresión. Cualquier modelo considerado debe ser un clasificador capaz de manejar más de dos clases.

2. Tipo de datos y modelos compatibles

El dataset de entrada es tabular, donde cada fila es una repetición de un ejercicio, y cada columna es una feature numérica continua (mínimo, máximo, rango, desviación estándar, media y mediana, calculadas sobre cada uno de los 12 canales de sensores). No hay variables categóricas ni texto, por lo que no se requiere ningún preprocesamiento especial al contar con únicamente variables numéricas continuas.

A modo de recapitulación previa, los canales `pitch`, `yaw` y `roll` corresponden a los ángulos de orientación de los sensores IMU donde se mide la inclinación, rotación de la extremidad y giro lateral. Mientras que los canales de `f1` a `f5` representan el ángulo de flexión de cada dedo captado por el guante sensorizado, describiendo el movimiento de la mano como el del brazo o la pierna.

Esta naturaleza de datos numéricos con un problema de clasificación multiclase, que a su vez no requiere de una estructura que obligue a utilizar un tipo específico de clasificador nos permite utilizar cualquier modelo de clasificación asociado a `scikit-learn`.

Nosotros utilizamos los siguientes:

- **KNN**: Clasifica por votación de los vecinos más cercanos
- **SVM**: Busca la frontera de decisión con mayor margen entre clases usando un kernel no lineal.
- **Regresión Logística**: Modelo lineal que puede aprender fronteras de decisión entre clases
- **Decision Tree**: Divide el espacio de features con reglas jerárquicas.
- **Random Forest**: Conjunto de árboles de decisión con votación, que a su vez lo hace más robusto.
- **Naive Bayes**: Modelo probabilístico que puede trabajar directamente con características numéricas

Estos modelos fueron seleccionados porque, además de aceptar entradas de dimensión fija, representan diferentes enfoques de clasificación, como los basados en distancia, en ensambles y en márgenes, entre otros. Esto nos permite realizar comparaciones de forma justa para saber cuál se adapta mejor a la naturaleza de nuestros datos.

La única distinción relevante es que los modelos basados en distancia o en gradiente (k-NN, SVM, regresión logística) requieren que las features estén en una escala comparable, mientras que los modelos como Decision Tree, Random Forest y Naive Bayes son indiferentes a la escala. Pero para mantener la simplicidad en el desarrollo, se les aplicó un escalamiento estándar por igual.

3. Dataset final

De los 16 ejercicios originales, el ejercicio 14 (knee flexion and extension) quedó excluido, pues de sus dos archivos .npy esperados solo uno estaba disponible, dejando ese ejercicio con 6 de los 12 canales necesarios. Al no poder construir el vector de 12 canales para ese ejercicio, se excluyó del dataset de entrenamiento, resultando en 15 clases utilizables en lugar de 16, siendo el enfoque seleccionado por nuestro equipo, al menos por el momento.

movement_id	status	n_samples
000	ok	232
001	ok	212
002	ok	267
003	ok	250
004	ok	287
005	ok	293
006	ok	260
007	ok	385
008	ok	299
009	ok	307
010	ok	311
011	ok	235
012	ok	293
013	ok	313
014	EXCLUDED	-
015	ok	313

Fig 1. Número de muestras por ejercicio, salvo en el ejercicio 14 el cual quedó excluido.

Por todo el dataset se cuentan con 4,257 muestras utilizables, distribuidas en 15 clases, con un desbalance moderado (212 a 385 muestras por ejercicio). Este desbalance es la razón por la que, más adelante, se usa F1-macro (y no solo accuracy) como criterio principal de comparación entre modelos.

4. Modelos investigados

Como se mencionó previamente, se seleccionaron 6 modelos clasificadores, elegidos por representar distintos enfoques para la clasificación, de forma que la comparación no esté sesgada hacia un solo tipo de lógica de decisión.

- *KNN*:
- *SVM*
- *Regresión Logística*
- *Decision Tree*
- *Random Forest*
- *Naive Bayes*

5. Métricas de evaluación

Para comparar los modelos se usaron cuatro métricas estándar de clasificación, calculadas de forma macro-promediada (el promedio simple de la métrica calculada por separado en cada una de las 15 clases, sin ponderar por su tamaño).

Esto es intencional puesto como se mencionó previamente, tenemos un desbalance moderado en nuestras clases, el macro-promediar obliga a que el modelo funcione bien en todas las clases por igual, tanto en las clases con mayor número de muestras como en las que menos muestras tienen.

- **Accuracy**: Proporción de predicciones correctas sobre el total.
- **Precision (macro)**: De las veces que el modelo predijo una clase, qué proporción realmente pertenecía a ella.
- **Recall (macro)**: De las muestras que realmente pertenecen a una clase, qué proporción fue detectada correctamente.
- **F1-score (macro)**: Media armónica entre precision y recall. Es la métrica principal de selección en este reporte, porque resume ambos errores en un solo número y es más robusta ante el desbalance entre ejercicios que la accuracy sola.

6. Métricas de evaluación

Los 6 modelos se evaluaron con validación cruzada estratificada de 5 folds, en lugar de un único split entrenamiento/prueba. Esto es importante por dos razones: primero, con 15 clases y un dataset moderado (4,257 muestras), un solo split del 20% podría, por azar, dejar mal representada alguna clase pequeña; segundo, la validación cruzada da una estimación más estable del desempeño real de cada modelo, en lugar de depender de una sola partición aleatoria. La estratificación asegura que la proporción de cada una de las 15 clases se mantenga en cada fold.

7. Resultados

Al realizar una comparación de métricas como f1, recall, precision, tiempo, etc., entre los modelos seleccionados se obtuvo los siguientes resultados.

```

Ranked by f1_macro:
  model      f1  accuracy  precision  recall  fit_time
random_forest 0.963570 0.964058 0.966230 0.963553 3.016256
svm_rbf      0.876258 0.879251 0.880352 0.877521 0.190552
logistic_regression 0.834650 0.837673 0.837461 0.835290 1.721453
knn          0.812024 0.813480 0.819252 0.815605 0.002953
decision_tree 0.805494 0.804554 0.827448 0.801974 0.121456
naive_bayes  0.701696 0.711768 0.731631 0.710767 0.007900
  
```

Fig 2. Resultados de la comparación entre modelos.

Random Forest y SVM (RBF) fueron los dos modelos con mejor F1-macro, con una diferencia considerable frente al resto, por esta razón identificamos que son los dos modelos más prometedores y por ende, nos centraremos en identificar si una determinada combinación de hiperparámetros podría mejorar estos modelos aún más. Random Forest tuvo, además, el mayor tiempo de entrenamiento (3.02s frente a 0.19s de SVM), lo cual consideramos es importante de mencionar puesto que hipotéticamente si la cantidad de muestras hubiera sido mayor, hubiera existido una gran diferencia en los tiempos de entrenamiento de estos dos, sin embargo, para el dataset que ya mencionamos de ~4000 muestras, no representa un mayor problema.

Cabe resaltar que no se hizo una combinación de hiperparámetros para todos los modelos porque hubiera sido extremadamente costoso en términos computacionales, puesto que identificamos que por cada modelo al que se le aplicaba este tuning solía tardar entre 2 minutos a 6 minutos.

8. Ajuste de hiper parámetros

Para la búsqueda de hiper parámetros se tuvieron las siguientes combinaciones:

Modelo	Hiper Parámetro	Valores
Random Forest	n_estimators (#trees)	100, 200, 300, 400, 500
Random Forest	max_depth	10, 20, sin límite
SVM (RBF)	C	0.1, 1, 10, 100
SVM (RBF)	gamma	'scale', 'auto', 0.001, 0.01, 0.1

9. Resultados de Random Forest: F1 vs #Trees

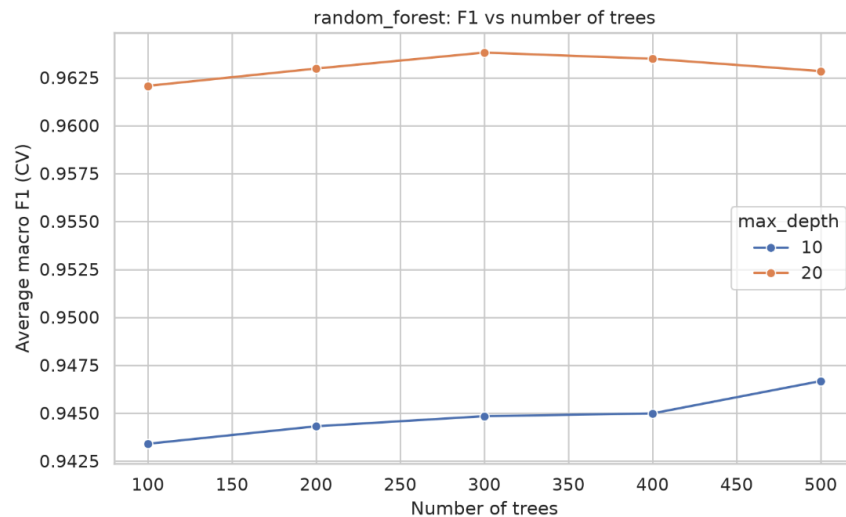


Fig 3. Comparación de diferentes cantidades de árboles con respecto al F1 macro

Se identificó que con depth=20 se supera consistentemente a depth=10 en cualquier número de árboles, sin ninguna excepción. Dentro de max_depth=20, el F1 alcanza su punto más alto en n_estimators=300 (~0.9635), a partir de ahí, agregar más árboles (400, 500) no mejora el desempeño e incluso lo reduce levemente. Por lo que la combinación ideal terminó siendo max depth de 20 y 300 árboles.

9. Resultados de SVM: F1 vs C

Nota: Donde el hiper parámetro C controla el equilibrio entre la amplitud del margen y la cantidad de errores de clasificación permitidos en los datos de entrenamiento

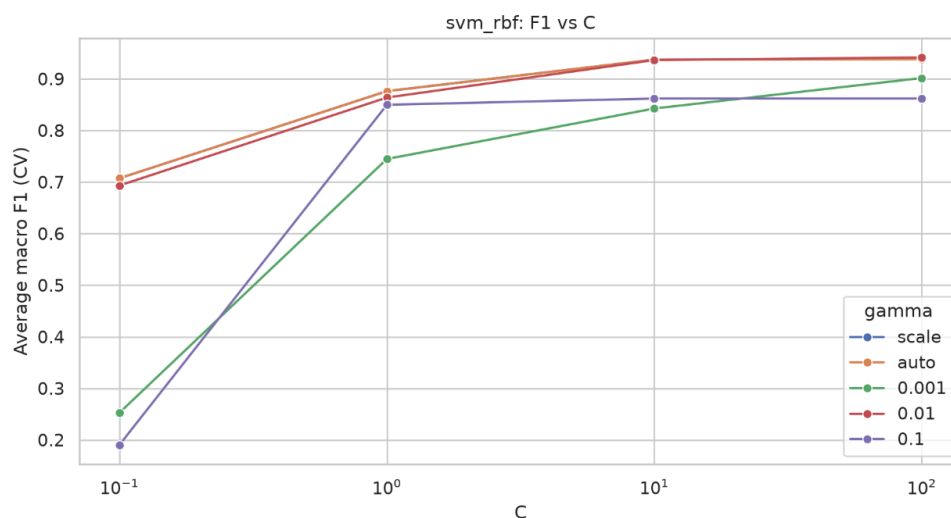


Fig 4. Comparación de diferentes valores de C con respecto al F1 macro

Se identificó que las curvas de scale y auto resultan prácticamente idénticas en todo el rango de C. Pero con respecto a C, el F1 mejora notablemente de C=0.1 a C=10, y se estabiliza después de ese punto, puesto que desde dicho valor de 10 hasta C=100 la ganancia es marginal y prácticamente imperceptible en la gráfica. Se evaluarán ambas y se decidió que C=10 sea el valor final junto a un valor de gamma de 'scale' porque da un valor de F1 prácticamente igual al de C=100 con un mucho menor riesgo de sobreajuste, por lo que el valor de C=10 es el definitivo.

10. Configuración final de los modelos.

Modelo	Hiper Parámetro final	F1-macro	Fit time
Random Forest	n_estimators (#trees)= 300, max_depth = 20	~0.964	~3.016s
SVM	C=10, gamma='scale'	~0.95	~0.2s

11. Conclusiones de selección final del modelo

Considerando la relación desempeño-tiempo, Random Forest ofrece el F1 más alto de los dos, pero a un costo de tiempo entrenamiento considerablemente mayor, sin embargo consideramos que puesto que no representa una cantidad de tiempo inaceptable o extremadamente notoria, nuestro equipo considera que Random Forest se selecciona como modelo final con los valores de hiperparametros previamente mencionados, sin embargo, si la cantidad de muestras fuera mayor, utilizar un SVM probaria ser muchisimo más eficiente en relación de desempeño tiempo.